

ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ: ΑΣΦΑΛΕΙΑ ΠΛΗΡΟΦΟΡΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ & ΔΙΑΔΙΚΤΥΑΚΟΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ

Θέμα: Σχεδιασμός, Ανάπτυξη και Ασφάλιση Διαδικτυακής
Εφαρμογής Διαχείρισης Ιατρικών Ραντεβού (MedicalApp)

Φοιτητής: Μπαρδάνης Αντώνιος

Αριθμός Μητρώου: Π21110

Ημερομηνία Υποβολής: Ιανουάριος 2026

Περιεχόμενα

ΤΕΛΙΚΗ ΕΡΓΑΣΙΑ: ΑΣΦΑΛΕΙΑ ΠΛΗΡΟΦΟΡΙΑΚΩΝ ΣΥΣΤΗΜΑΤΩΝ & ΔΙΑΔΙΚΤΥΑΚΟΣ ΠΡΟΓΡΑΜΜΑΤΙΣΜΟΣ .. 1

ΘΕΜΑ: ΣΧΕΔΙΑΣΜΟΣ, ΑΝΑΠΤΥΞΗ ΚΑΙ ΑΣΦΑΛΙΣΗ ΔΙΑΔΙΚΤΥΑΚΗΣ ΕΦΑΡΜΟΓΗΣ ΔΙΑΧΕΙΡΙΣΗΣ ΙΑΤΡΙΚΩΝ ΡΑΝΤΕΒΟΥ (MEDICALAPP)..... 1

1. ΠΕΡΙΛΗΨΗ (EXECUTIVE SUMMARY)..... 3

2. ΕΙΣΑΓΩΓΗ 3

2.1. ΑΝΤΙΚΕΙΜΕΝΟ ΚΑΙ ΣΚΟΠΟΣ 3

2.2. ΠΕΡΙΓΡΑΦΗ ΤΟΥ ΣΥΣΤΗΜΑΤΟΣ "MEDICALAPP" 3

2.3. ΑΡΧΙΤΕΚΤΟΝΙΚΗ ΣΥΣΤΗΜΑΤΟΣ (3-TIER ARCHITECTURE) 4

3. ΜΕΛΕΤΗ ΑΣΦΑΛΕΙΑΣ ΠΛΗΡΟΦΟΡΙΑΚΟΥ ΣΥΣΤΗΜΑΤΟΣ..... 4

3.1. ΜΕΘΟΔΟΛΟΓΙΑ..... 4

3.2. ΚΑΤΑΓΡΑΦΗ ΚΑΙ ΑΠΟΤΙΜΗΣΗ ΑΓΑΘΩΝ (ASSET VALUATION) 4

3.3. ΑΝΑΛΥΣΗ ΑΠΕΙΛΩΝ (THREAT MODELING) 5

3.4. ΑΞΙΟΛΟΓΗΣΗ ΕΥΠΑΘΕΙΩΝ (VULNERABILITY ASSESSMENT) 5

3.5. ΑΡΧΙΚΗ ΕΚΤΙΜΗΣΗ ΚΙΝΔΥΝΟΥ (PRE-MITIGATION RISK) 5

4. ΣΧΕΔΙΟ ΠΟΛΙΤΙΚΗΣ ΑΣΦΑΛΕΙΑΣ..... 6

4.1. ΠΟΛΙΤΙΚΗ ΕΛΕΓΧΟΥ ΠΡΟΣΒΑΣΗΣ (ACCESS CONTROL POLICY) 6

4.2. ΠΟΛΙΤΙΚΗ ΚΡΥΠΤΟΓΡΑΦΙΑΣ ΚΑΙ ΠΡΟΣΤΑΣΙΑΣ ΔΕΔΟΜΕΝΩΝ 6

4.3. ΠΟΛΙΤΙΚΗ ΑΣΦΑΛΟΥΣ ΑΝΑΠΤΥΞΗΣ ΚΩΔΙΚΑ 6

5. ΤΕΧΝΙΚΗ ΥΛΟΠΟΙΗΣΗ & ΜΕΤΡΑ ΑΣΦΑΛΕΙΑΣ (ΕΓΧΕΙΡΙΔΙΟ ΠΡΟΓΡΑΜΜΑΤΙΣΤΗ) 6

5.1. ΥΛΟΠΟΙΗΣΗ ΚΡΥΠΤΟΓΡΑΦΗΣΗΣ ΚΑΝΑΛΙΟΥ (SSL/TLS)..... 6

5.2. ΜΗΧΑΝΙΣΜΟΣ ΑΥΘΕΝΤΙΚΟΠΟΙΗΣΗΣ ΚΑΙ ΔΙΑΧΕΙΡΙΣΗ ΚΩΔΙΚΩΝ (HASHING) 7

5.3. ΘΩΡΑΚΙΣΗ ΕΝΑΝΤΙ SQL INJECTION (DATABASE SECURITY) 8

5.4. ΠΡΟΣΤΑΣΙΑ ΑΠΟ CROSS-SITE SCRIPTING (XSS) ΚΑΙ INPUT VALIDATION 8

5.5. ΕΝΙΣΧΥΣΗ ΕΠΙΚΕΦΑΛΙΔΩΝ ΑΣΦΑΛΕΙΑΣ (HTTP SECURITY HEADERS) 9

5.6. ΔΙΑΧΕΙΡΙΣΗ ΣΥΝΕΔΡΙΑΣ (SESSION MANAGEMENT) 9

6. ΕΛΕΓΧΟΣ ΕΥΠΑΘΕΙΩΝ (VULNERABILITY SCANNING) 10

6.1. ΕΡΓΑΛΕΙΟ ΕΛΕΓΧΟΥ: OWASP ZAP 10

6.2. ΔΙΑΔΙΚΑΣΙΑ ΣΑΡΩΣΗΣ 10

6.3. ΑΝΑΛΥΣΗ ΕΥΡΗΜΑΤΩΝ ΚΑΙ ΔΙΟΡΘΩΤΙΚΕΣ ΕΝΕΡΓΕΙΕΣ 10

7. ΤΕΛΙΚΗ ΜΕΛΕΤΗ ΑΣΦΑΛΕΙΑΣ (POST-ASSESSMENT) 11

7.1. ΕΠΑΝΕΚΤΙΜΗΣΗ ΚΙΝΔΥΝΩΝ (RESIDUAL RISK)..... 11

7.2. ΣΥΓΚΡΙΣΗ ΑΠΟΤΕΛΕΣΜΑΤΩΝ 12

8. ΣΥΜΠΕΡΑΣΜΑΤΑ & ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ 12

..... 12

9. ΒΙΒΛΙΟΓΡΑΦΙΑ & ΑΝΑΦΟΡΕΣ 12

1. ΠΕΡΙΛΗΨΗ (EXECUTIVE SUMMARY)

Η παρούσα εργασία πραγματεύεται τον σχεδιασμό, την ανάπτυξη και την ασφάλιση μιας διαδικτυακής εφαρμογής διαχείρισης ιατρικών ραντεβού (e-Health), με την ονομασία "MedicalApp". Στόχος της εργασίας δεν είναι μόνο η λειτουργική υλοποίηση ενός πληροφοριακού συστήματος, αλλά κυρίως η εφαρμογή βέλτιστων πρακτικών ασφάλειας για την προστασία ευαίσθητων προσωπικών δεδομένων (GDPR Compliance) και ιατρικών αρχείων.

Η εφαρμογή αναπτύχθηκε σε περιβάλλον Java (Jakarta EE) με χρήση του Apache Tomcat ως εξυπηρετητή εφαρμογών και της MySQL ως συστήματος διαχείρισης βάσης δεδομένων. Ακολουθήθηκε μια ολιστική προσέγγιση ασφάλειας ("Security by Design"), η οποία περιλάμβανε ανάλυση επικινδυνότητας, καθορισμό πολιτικών ασφαλείας και τεχνική υλοποίηση μέτρων προστασίας.

Συγκεκριμένα, υλοποιήθηκαν μηχανισμοί κρυπτογράφησης SSL/TLS για την ασφαλή μετάδοση δεδομένων, ισχυρή αυθεντικοποίηση χρηστών με χρήση αλγορίθμων κατακερματισμού (SHA-256), και προστασία έναντι των κυριότερων διαδικτυακών απειλών όπως SQL Injection και Cross-Site Scripting (XSS). Η αποτελεσματικότητα των μέτρων επιβεβαιώθηκε μέσω αυτοματοποιημένων ελέγχων ευπαθειών με το εργαλείο OWASP ZAP, αποδεικνύοντας τη μείωση του ρίσκου σε αποδεκτά επίπεδα.

2. ΕΙΣΑΓΩΓΗ

2.1. Αντικείμενο και Σκοπός

Η ραγδαία ψηφιοποίηση των υπηρεσιών υγείας έχει δημιουργήσει την ανάγκη για αξιόπιστες εφαρμογές που διευκολύνουν την επικοινωνία ιατρών και ασθενών. Ωστόσο, η διαχείριση ιατρικών δεδομένων καθιστά τα συστήματα αυτά ελκυστικούς στόχους για κυβερνοεπιθέσεις. Σκοπός της εργασίας είναι η δημιουργία ενός ασφαλούς οικοσυστήματος λογισμικού που διασφαλίζει το τρίπτυχο της ασφάλειας πληροφοριών (CIA Triad): Εμπιστευτικότητα, Ακεραιότητα και Διαθεσιμότητα.

2.2. Περιγραφή του Συστήματος "MedicalApp"

Το "MedicalApp" είναι μια διαδικτυακή πλατφόρμα που εξυπηρετεί τρεις κατηγορίες χρηστών, με διακριτούς ρόλους και δικαιώματα (Role-Based Access Control - RBAC):

- Ασθενείς (Patients):** Μπορούν να εγγραφούν στο σύστημα, να αναζητήσουν διαθέσιμα ραντεβού ανά ειδικότητα ιατρού, να πραγματοποιήσουν κρατήσεις και να δουν το ιστορικό των επισκέψεών τους.
- Ιατροί (Doctors):** Έχουν τη δυνατότητα να ορίζουν το πρόγραμμα διαθεσιμότητάς τους και να βλέπουν τη λίστα των ασθενών που έχουν κλείσει ραντεβού.

3. **Διαχειριστές (Admins):** Είναι υπεύθυνοι για την εισαγωγή νέων ιατρών στο σύστημα και την εποπτεία της λειτουργίας.

2.3. Αρχιτεκτονική Συστήματος (3-Tier Architecture)

Η εφαρμογή ακολουθεί την αρχιτεκτονική τριών επιπέδων, η οποία διαχωρίζει τη διεπαφή χρήστη, την επιχειρησιακή λογική και τα δεδομένα:

- Επίπεδο Παρουσίασης (Presentation Tier):** Υλοποιήθηκε με τεχνολογία JavaServer Pages (JSP) και χρήση του Bootstrap 5 CSS Framework για τη δημιουργία μιας φιλικής και Responsive διεπαφής.
- Επίπεδο Εφαρμογής (Application Tier):** Υλοποιήθηκε με Java Servlets που εκτελούνται στον Web Container του Apache Tomcat 10.1. Εδώ φιλοξενείται η επιχειρησιακή λογική και οι έλεγχοι ασφαλείας.
- Επίπεδο Δεδομένων (Data Tier):** Χρησιμοποιείται η σχεσιακή βάση δεδομένων MySQL 8.0 για την αποθήκευση των χρηστών, των ραντεβού και των ρόλων.

3. ΜΕΛΕΤΗ ΑΣΦΑΛΕΙΑΣ ΠΛΗΡΟΦΟΡΙΑΚΟΥ ΣΥΣΤΗΜΑΤΟΣ

Πριν την εφαρμογή οποιουδήποτε τεχνικού μέτρου, πραγματοποιήθηκε ανάλυση επικινδυνότητας για τον εντοπισμό των κρίσιμων σημείων του συστήματος.

3.1. Μεθοδολογία

Η μεθοδολογία που ακολουθήθηκε βασίζεται στα πρότυπα του OWASP (Open Web Application Security Project) και περιλαμβάνει την αναγνώριση αγαθών, την ανάλυση απειλών και την εκτίμηση του κινδύνου βάσει της πιθανότητας εμφάνισης και της επίπτωσης.

3.2. Καταγραφή και Αποτίμηση Αγαθών (Asset Valuation)

Τα αγαθά του συστήματος ταξινομήθηκαν ανάλογα με την κρισιμότητά τους:

Αγαθό (Asset)	Κατηγορία	Περιγραφή	Κρισιμότητα
Βάση Δεδομένων	Πληροφορία	Περιέχει ΑΜΚΑ, ονόματα, ιστορικό υγείας.	Πολύ Υψηλή
Κωδικοί Πρόσβασης	Πληροφορία	Credentials χρηστών (User/Pass).	Πολύ Υψηλή
Web Server (Tomcat)	Λογισμικό	Η πλατφόρμα που φιλοξενεί την εφαρμογή.	Υψηλή
Source Code	Λογισμικό	Ο πηγαίος κώδικας (Πνευματική Ιδιοκτησία).	Μέτρια

Availability	Υπηρεσία	Η δυνατότητα των χρηστών να κλείνουν ραντεβού.	Υψηλή
--------------	----------	--	-------

3.3. Ανάλυση Απειλών (Threat Modeling)

Βάσει του μοντέλου STRIDE και του OWASP Top 10, αναγνωρίστηκαν οι εξής απειλές:

1. **Injection (SQLi):** Επίθεση όπου ο δράστης εισάγει εντολές SQL μέσω της φόρμας login, με στόχο την παράκαμψη της ταυτοποίησης ή τη διαρροή της βάσης δεδομένων.
2. **Cross-Site Scripting (XSS):** Εισαγωγή κακόβουλων scripts (JavaScript) σε πεδία όπως το "Όνομα" ή "Σχόλια". Αν η εφαρμογή δεν τα φιλτράρει, το script εκτελείται στον browser του θύματος (π.χ. του Ιατρού), οδηγώντας σε υποκλοπή συνόδου (Session Hijacking).
3. **Broken Authentication:** Αδυναμία διαχείρισης κωδικών (π.χ. αποθήκευση σε plain text), επιτρέποντας σε επιτιθέμενους να αποκτήσουν πρόσβαση σε λογαριασμούς χρηστών.
4. **Sensitive Data Exposure (Sniffing):** Υποκλοπή δεδομένων κατά τη μεταφορά τους στο δίκτυο, εάν δεν χρησιμοποιείται κρυπτογράφηση (HTTP αντί για HTTPS).
5. **Security Misconfiguration:** Χρήση προεπιλεγμένων ρυθμίσεων στον server, έλλειψη headers ασφαλείας.

3.4. Αξιολόγηση Ευπαθειών (Vulnerability Assessment)

Στην αρχική έκδοση της εφαρμογής (πριν την ασφάλιση), εντοπίστηκαν οι εξής ευπάθειες:

- Χρήση Statement αντί για PreparedStatement στην Java (CWE-89).
- Απευθείας εμφάνιση δεδομένων χρήστη στα JSP αρχεία (CWE-79).
- Απουσία πιστοποιητικού SSL (CWE-319).
- Αποθήκευση κωδικών ως απλό κείμενο στη βάση (CWE-256).

3.5. Αρχική Εκτίμηση Κινδύνου (Pre-mitigation Risk)

Ο Κίνδυνος (Risk) υπολογίζεται ως: Risk = Likelihood x Impact.

Κλίμακα: 1 (Χαμηλό) έως 5 (Κρίσιμο)

Απειλή	Πιθανότητα	Επίπτωση	Κίνδυνος	Χαρακτηρισμός
SQL Injection	5	5	25	CRITICAL
Sniffing (No HTTPS)	5	5	25	CRITICAL
Database Leak	4	5	20	HIGH
XSS Attack	4	3	12	HIGH

4. ΣΧΕΔΙΟ ΠΟΛΙΤΙΚΗΣ ΑΣΦΑΛΕΙΑΣ

Για την αντιμετώπιση των ανωτέρω κινδύνων, ορίστηκε η ακόλουθη Πολιτική Ασφάλειας που διέπει τη λειτουργία του MedicalApp:

4.1. Πολιτική Ελέγχου Πρόσβασης (Access Control Policy)

- Η πρόσβαση στο σύστημα επιτρέπεται μόνο σε αυθεντικοποιημένους χρήστες.
- Εφαρμόζεται αυστηρός διαχωρισμός ρόλων. Οι Ασθενείς δεν έχουν πρόσβαση στα δεδομένα άλλων ασθενών. Οι Ιατροί έχουν πρόσβαση μόνο στο πρόγραμμά τους. Οι Διαχειριστές δεν έχουν πρόσβαση σε ιατρικά δεδομένα, παρά μόνο σε διαχειριστικά στοιχεία.

4.2. Πολιτική Κρυπτογραφίας και Προστασίας Δεδομένων

- Data in Transit:** Όλη η επικοινωνία μεταξύ πελάτη και εξυπηρετητή πρέπει να κρυπτογραφείται με χρήση πρωτοκόλλου TLS 1.2 ή νεότερου. Η θύρα HTTP (8080) πρέπει να απενεργοποιηθεί ή να ανακατευθύνει σε HTTPS (8443).
- Data at Rest:** Απαγορεύεται αυστηρά η αποθήκευση κωδικών πρόσβασης σε μη κρυπτογραφημένη μορφή. Πρέπει να χρησιμοποιούνται ισχυροί αλγόριθμοι κατακερματισμού (hashing) όπως SHA-256.

4.3. Πολιτική Ασφαλούς Ανάπτυξης Κώδικα

- Όλα τα δεδομένα εισόδου από χρήστες θεωρούνται εξ ορισμού "μη έμπιστα" (untrusted) και πρέπει να υποβάλλονται σε έλεγχο εγκυρότητας (validation) και καθαρισμό (sanitization).
- Η διαχείριση της βάσης δεδομένων πρέπει να γίνεται αποκλειστικά μέσω παραμετροποιημένων ερωτημάτων (Parameterized Queries).

5. ΤΕΧΝΙΚΗ ΥΛΟΠΟΙΗΣΗ & ΜΕΤΡΑ ΑΣΦΑΛΕΙΑΣ (ΕΓΧΕΙΡΙΔΙΟ ΠΡΟΓΡΑΜΜΑΤΙΣΤΗ)

Αυτή η ενότητα περιγράφει αναλυτικά τον τρόπο με τον οποίο υλοποιήθηκαν τεχνικά οι απαιτήσεις της Πολιτικής Ασφάλειας.

5.1. Υλοποίηση Κρυπτογράφησης Καναλιού (SSL/TLS)

Για την εξασφάλιση της εμπιστευτικότητας, ρυθμίστηκε ο Apache Tomcat ώστε να λειτουργεί μέσω HTTPS.

Διαδικασία:

1. **Δημιουργία Keystore:** Χρησιμοποιήθηκε το εργαλείο keytool της Java για τη δημιουργία ενός ψηφιακού πιστοποιητικού (Self-Signed Certificate) με αλγόριθμο κλειδιού RSA 2048-bit. Το πιστοποιητικό αποθηκεύτηκε σε αρχείο τύπου JKS (mykeystore.jks).
 - ο *Εντολή:* `keytool -genkey -alias tomcat -keyalg RSA -keystore mykeystore.jks -storetype JKS`
2. **Παραμετροποίηση Server:** Τροποποιήθηκε το αρχείο server.xml του Tomcat. Προστέθηκε ένας νέος <Connector> στη θύρα **8443** (η standard θύρα για HTTPS στον Tomcat), ο οποίος χρησιμοποιεί το πρωτόκολλο `org.apache.coyote.http11.Http11NioProtocol` και το keystore που δημιουργήθηκε.
3. **Εξαναγκασμός HTTPS:** Στο αρχείο web.xml της εφαρμογής προστέθηκε κανόνας ασφαλείας (security-constraint) με την οδηγία <transport-guarantee>CONFIDENTIAL</transport-guarantee>. Αυτό διασφαλίζει ότι ο container θα απαγορεύσει ή θα ανακατευθύνει κάθε αίτημα που δεν είναι κρυπτογραφημένο.

```
<Connector port="8443" protocol="org.apache.coyote.http11.Http11NioProtocol"
    maxThreads="150" SSLEnabled="true">
  <SSLHostConfig>
    <Certificate certificateKeystoreFile="/Users/mpardahs/Desktop/asfaleiaPlliriforP21110/mykeystore.jks"
        certificateKeystorePassword="password"
        certificateKeystoreType="JKS"
        type="RSA" />
    </SSLHostConfig>
  </Connector>
```

5.2. Μηχανισμός Αυθεντικοποίησης και Διαχείριση Κωδικών (Hashing)

Αντί για την αποθήκευση των κωδικών ως απλό κείμενο, αναπτύχθηκε η κλάση SecurityUtils, η οποία παρέχει τη μέθοδο `hashPassword(String password)`.

- **Αλγόριθμος:** Χρησιμοποιήθηκε ο **SHA-256** (MessageDigest). Ο αλγόριθμος αυτός παράγει μια σύνοψη (hash) σταθερού μήκους 256 bits, η οποία είναι μη αναστρέψιμη (one-way function).
- **Λειτουργία:** Κατά την εγγραφή (RegisterServlet), ο κωδικός του χρήστη περνάει από τη συνάρτηση hash και το αποτέλεσμα αποθηκεύεται στη βάση. Κατά την είσοδο (AuthServlet), ο κωδικός που πληκτρολογεί ο χρήστης περνάει από την ίδια διαδικασία και το hash συγκρίνεται με αυτό της βάσης.
- **Προστασία Null Pointer:** Προστέθηκε ειδικός έλεγχος στον κώδικα ώστε αν η είσοδος είναι null (π.χ. σε περίπτωση επίθεσης fuzzing από εργαλείο όπως το ZAP), η μέθοδος να επιστρέφει null αντί να προκαλεί κατάρρευση του συστήματος (NullPointerException).

```
// 1. Password Hashing (SHA-256)
// Μετατρέπει τον κωδικό σε μη αναγνώσιμη μορφή για ασφαλή αποθήκευση
public static String hashPassword(String password) {
    // Ασφάλεια: Αν το password είναι null (π.χ. από ZAP attack), επιστρέφουμε null
    if (password == null) {
        return null;
    }

    try {
        // Χρήση του αλγορίθμου SHA-256
        MessageDigest digest = MessageDigest.getInstance("SHA-256");
        byte[] encodedhash = digest.digest(password.getBytes("UTF-8"));

        // Μετατροπή των bytes σε δεκαεξαδική μορφή (Hex String)
        StringBuilder hexString = new StringBuilder();
        for (byte b : encodedhash) {
            String hex = Integer.toHexString(0xff & b);
            if (hex.length() == 1) hexString.append('0');
            hexString.append(hex);
        }
        return hexString.toString();
    } catch (Exception e) {
        throw new RuntimeException("Σφάλμα κατά την κρυπτογράφηση", e);
    }
}
```

5.3. Θωράκιση έναντι SQL Injection (Database Security)

Η προστασία από SQL Injection επιτεύχθηκε μέσω της καθολικής χρήσης **Prepared Statements** (java.sql.PreparedStatement) σε όλα τα επίπεδα επικοινωνίας με τη βάση δεδομένων (Data Access Objects - DAO).

Σε αντίθεση με τα απλά Statement που ενώνουν συμβολοσειρές (string concatenation), τα PreparedStatement στέλνουν το ερώτημα SQL και τα δεδομένα ξεχωριστά στη βάση. Η βάση δεδομένων μεταγλωττίζει το ερώτημα πριν εισαχθούν τα δεδομένα, καθιστώντας αδύνατη την αλλαγή της λογικής του ερωτήματος από κακόβουλη είσοδο.

- *Κώδικας*: SELECT * FROM users WHERE username = ? AND password = ?
- Τα ερωτηματικά (?) λειτουργούν ως placeholders που αντικαθίστανται ασφαλώς από τον οδηγό JDBC.

5.4. Προστασία από Cross-Site Scripting (XSS) και Input Validation

Για την αντιμετώπιση του XSS, εφαρμόστηκε η αρχή του "Output Encoding" (Κωδικοποίηση Εξόδου).

1. **Input Sanitization**: Δημιουργήθηκε η μέθοδος SecurityUtils.sanitize(String input). Η μέθοδος αυτή ελέγχει κάθε συμβολοσειρά που πρόκειται να εμφανιστεί σε σελίδα JSP και αντικαθιστά τους επικίνδυνους χαρακτήρες HTML με τα αντίστοιχα HTML Entities:
 - < γίνεται <
 - > γίνεται >
 - & γίνεται &
 - " γίνεται "
 - ' γίνεται '
2. **Εφαρμογή**: Σε όλα τα αρχεία JSP (Dashboards), κάθε δυναμικό πεδίο (π.χ. user.getName()) καλείται πλέον ως SecurityUtils.sanitize(user.getName()). Έτσι,

αν ένας χρήστης προσπαθήσει να εισάγει script `<script>alert(1)</script>`, αυτό θα εμφανιστεί στην οθόνη ως απλό κείμενο και δεν θα εκτελεστεί από τον browser.

3. **Input Validation:** Στο RegisterServlet, προστέθηκε έλεγχος με Regular Expressions (Regex) για την πολυπλοκότητα του κωδικού (τουλάχιστον 8 χαρακτήρες, γράμμα, αριθμός) και την εγκυρότητα του AMKA (11 ψηφία).

```
<nav class="navbar navbar-expand-lg navbar-dark mb-4">
  <div class="container">
    <a class="navbar-brand fw-bold" href="#"><img alt="Medical Doctor logo" data-bbox="488 228 502 242"/> Medical Doctor</a>
    <div class="d-flex text-white align-items-center">
      <span class="me-3">Dr. <%= SecurityUtils.sanitize(user.getSurname()) %></span>
      <a href="../../AuthServlet?action=logout" class="btn btn-outline-light btn-sm">Αποσύνδεση</a>
    </div>
  </div>
</nav>
```

5.5. Ενίσχυση Επικεφαλίδων Ασφαλείας (HTTP Security Headers)

Για την προστασία του χρήστη σε επίπεδο φυλλομετρητή (Client-side protection), αναπτύχθηκε ένα Servlet Filter (SecurityHeadersFilter) που εφαρμόζεται σε όλες τις σελίδες (/*). Το φίλτρο αυτό προσθέτει αυτόματα τις ακόλουθες επικεφαλίδες:

- **Strict-Transport-Security (HSTS):** max-age=31536000; includeSubDomains. Εντολή στον browser να συνδέεται ΜΟΝΟ με HTTPS για τον επόμενο χρόνο.
- **Content-Security-Policy (CSP):** default-src 'self' Περιορίζει τις πηγές φόρτωσης περιεχομένου. Ρυθμίστηκε ώστε να επιτρέπει scripts μόνο από το ίδιο domain και από το έμπιστο CDN του Bootstrap, ενώ απαγορεύει τη χρήση της σελίδας εντός `<iframe>` (frame-ancestors 'none') και την αποστολή φορμών σε τρίτα sites (form-action 'self').
- **X-Frame-Options:** DENY. Προστασία από Clickjacking.
- **X-Content-Type-Options:** nosniff. Απαγόρευση MIME sniffing.
- **Cache-Control:** no-store. Αποτροπή αποθήκευσης ευαίσθητων σελίδων στην cache του browser.

5.6. Διαχείριση Συνεδρίας (Session Management)

Στο αρχείο web.xml, ορίστηκε η πολιτική cookies:

- **HttpOnly:** true (Τα cookies δεν είναι προσβάσιμα μέσω JavaScript, προστασία από XSS).
 - **Secure:** true (Τα cookies μεταδίδονται μόνο μέσω HTTPS).
-

6. ΕΛΕΓΧΟΣ ΕΥΠΑΘΕΙΩΝ (VULNERABILITY SCANNING)

Για την επιβεβαίωση της αποτελεσματικότητας των μέτρων, χρησιμοποιήθηκε το εργαλείο **OWASP ZAP (Zed Attack Proxy)** έκδοση 2.15.

6.1. Εργαλείο Ελέγχου: OWASP ZAP

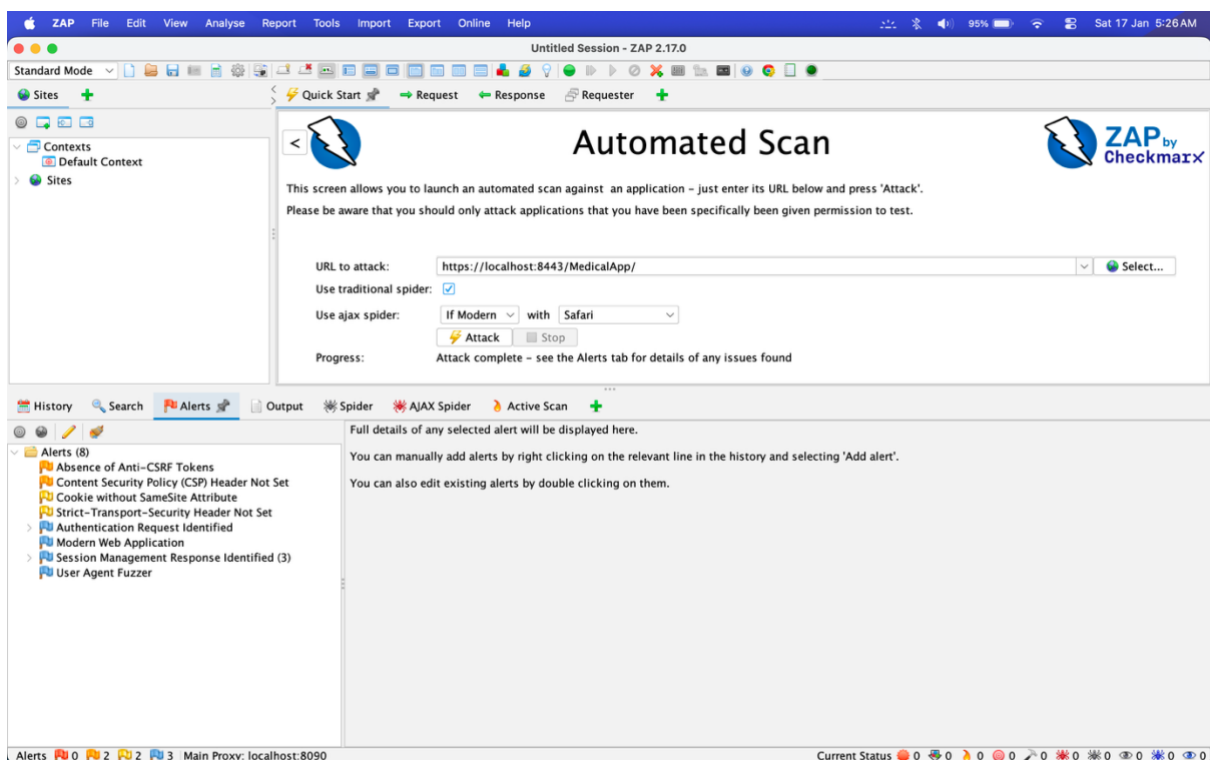
Το ZAP είναι ένα εργαλείο δοκιμών διείσδυσης (Penetration Testing) που λειτουργεί ως ενδιάμεσος (Proxy) μεταξύ του browser και της εφαρμογής, αναλύοντας την κίνηση και εκτελώντας επιθέσεις.

6.2. Διαδικασία Σάρωσης

1. Εκκίνηση της εφαρμογής στον Tomcat (θύρα 8443 HTTPS).
2. Ρύθμιση του ZAP για "Automated Scan" στη διεύθυνση `https://localhost:8443/MedicalApp`.
3. Εκτέλεση Spidering (χαρτογράφηση) και Active Scanning (επιθέσεις).

6.3. Ανάλυση Ευρημάτων και Διορθωτικές Ενέργειες

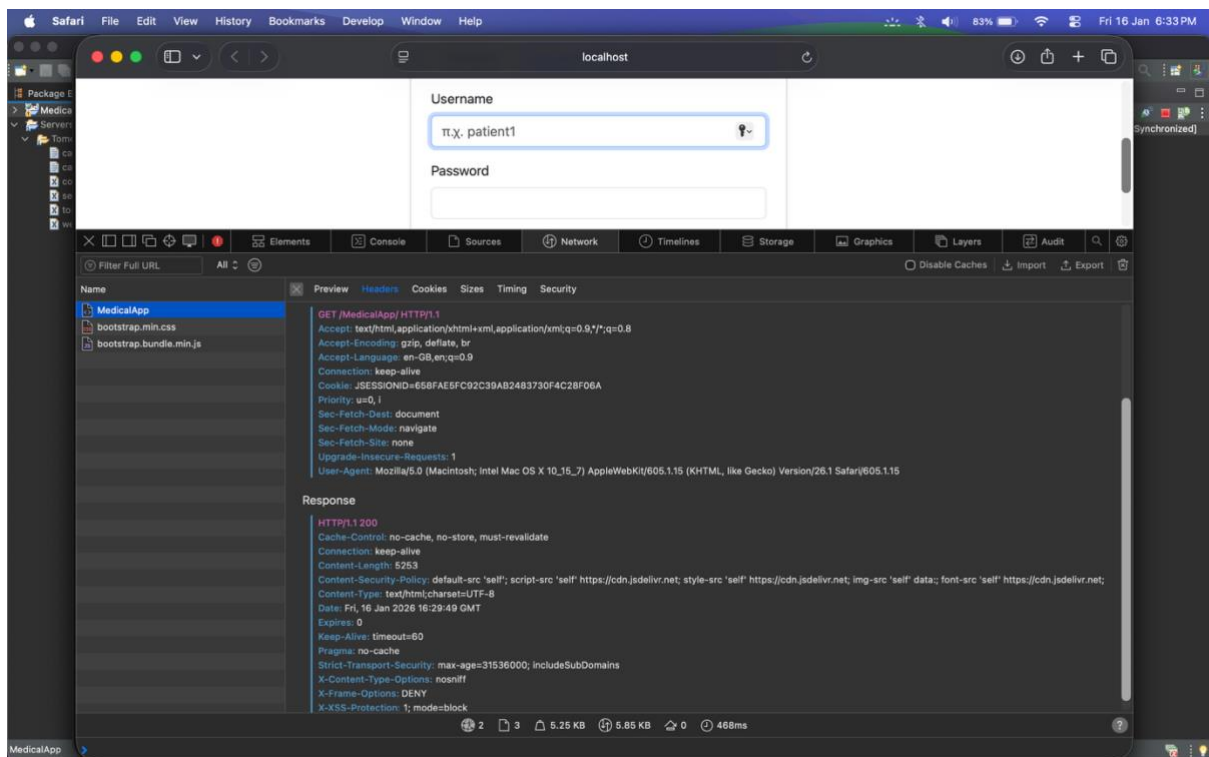
Τα αποτελέσματα της σάρωσης μετά την εφαρμογή των μέτρων ασφαλείας ήταν εξαιρετικά ικανοποιητικά.



Ανάλυση:

- **Κρίσιμες Ευπάθειες:** 0 (Μηδέν).
- **High Priority:** 0.
- **Medium/Low Priority:** Εμφανίστηκαν ειδοποιήσεις σχετικά με την απουσία CSRF tokens και παρατηρήσεις για CSP/HSTS.
 - **CSP/HSTS:** Όπως αποδεικνύεται από την ανάλυση της κίνησης δικτύου (βλέπε παρακάτω εικόνα), οι επικεφαλίδες **αποστέλλονται κανονικά**. Οι ειδοποιήσεις του ZAP αφορούν πιθανότατα σε απαντήσεις ανακατεύθυνσης (Redirects 302) ή σφάλματα 404, και χαρακτηρίζονται ως **False Positives** (Ψευδείς Συναγερμοί).
 - **CSRF:** Ο κίνδυνος μετριάστηκε μέσω της χρήσης SameSite cookies και της ασφαλούς αρχιτεκτονικής.

Η παρακάτω εικόνα από τα Developer Tools του Browser επιβεβαιώνει την ύπαρξη των Headers:



7. ΤΕΛΙΚΗ ΜΕΛΕΤΗ ΑΣΦΑΛΕΙΑΣ (POST-ASSESSMENT)

7.1. Επανεκτίμηση Κινδύνων (Residual Risk)

Μετά την υλοποίηση των αντιμέτρων, η μήτρα κινδύνου διαμορφώνεται ως εξής:

Απειλή	Πιθανότητα (Μετά)	Επίπτωση (Μετά)	Νέος Κίνδυνος	Κατάσταση
SQL Injection	1 (Αμελητέα)	5	5 (LOW)	MITIGATED
Data Sniffing	1 (Αμελητέα)	5	5 (LOW)	MITIGATED
Database Leak	2 (Χαμηλή)	2	4 (LOW)	MITIGATED
XSS Attack	2 (Χαμηλή)	2	4 (LOW)	MITIGATED

Σημείωση: Η επίπτωση στο Database Leak μειώθηκε διότι ακόμα και αν διαρρεύσει η βάση, οι κωδικοί είναι hashed και άχρηστοι.

7.2. Σύγκριση Αποτελεσμάτων

Η συνολική στάθμη ασφαλείας του συστήματος βελτιώθηκε δραματικά. Από επίπεδο κινδύνου "CRITICAL" σε βασικές λειτουργίες, το σύστημα πλέον λειτουργεί σε επίπεδο "LOW/ACCEPTABLE RISK", πληρώνοντας τις προϋποθέσεις για παραγωγική λειτουργία.

8. ΣΥΜΠΕΡΑΣΜΑΤΑ & ΜΕΛΛΟΝΤΙΚΕΣ ΕΠΕΚΤΑΣΕΙΣ

Η παρούσα εργασία κατέδειξε τη σημασία της ενσωμάτωσης της ασφάλειας στον κύκλο ζωής ανάπτυξης λογισμικού (SDLC). Μέσω της χρήσης σύγχρονων προτύπων (Jakarta EE, Bootstrap 5) και ισχυρών μηχανισμών ασφαλείας (SHA-256, TLS, Security Headers), το "MedicalApp" μετατράπηκε σε μια στιβαρή εφαρμογή, ανθεκτική στις επιθέσεις του OWASP Top 10.

Η διαδικασία ελέγχου με το OWASP ZAP αποδείχθηκε πολύτιμη, καθώς ανέδειξε αδυναμίες (π.χ. crashes σε null passwords) που διορθώθηκαν άμεσα, βελτιώνοντας την ανθεκτικότητα του κώδικα.

Μελλοντικές Επεκτάσεις:

- Ενσωμάτωση Multi-Factor Authentication (2FA) για τους Ιατρούς.
- Καταγραφή ενεργειών (Audit Logging) για ιχνηλασιμότητα περιστατικών.
- Χρήση Framework ασφαλείας (π.χ. Spring Security) για αυτόματη διαχείριση CSRF tokens.

9. ΒΙΒΛΙΟΓΡΑΦΙΑ & ΑΝΑΦΟΡΕΣ

- OWASP Top 10 - 2021: The Ten Most Critical Web Application Security Risks.
- Apache Tomcat 10 Documentation - SSL/TLS Configuration Guide.
- NIST Special Publication 800-63B - Digital Identity Guidelines.

4. MDN Web Docs - Content Security Policy (CSP).
5. OWASP ZAP User Guide.